2 计划 (plan.cpp/in/out)

NOIP2017 已经开始考树状数组了，之前也已经多次考察了离线的方法，于是我出了这道可以离线后用树状数组解决的题。

2.1 算法一

根据题意暴力，根据实现不同时间复杂度为 $\Theta(qn^3)$ 或 $\Theta(qn^2)$。

2.2 算法二

可以用区间 DP 来预处理出所有区间的答案。

首先介绍一下常用的处理区间不同数的方法，先求出 i 的前驱 u_i 表示从 i 向前最近的满足 $a_j = a_i$ 的 j（若不存在则为 0），那么区间 $[l,\ r]$ 不同数的个数等于 $l \le i \le r$ 且 $u_i < l$ 的 i 的个数，相当于每个数只会在第一次出现时被统计到。

记 $e[i,\ j]$ 表示区间 $[i,\ j]$ 中不同数的个数，那么（其中方括号 [cond] 表示判断，即 cond 为真时值为 1，否则为 0）

$$e[i\ j] = e[i,\ j-1] + [u_j < i]$$

再记 $f[i,\ j]$ 表示区间 $[i,\ j]$ 的答案，那么除 $[i,\ j]$ 本身外的所有子区间的答案和为 $f[i,\ j-1] + f[i+1,\ j] - f[i+1,\ j-1]$（其中 $[i+1,\ j-1]$ 的所有子区间的被算了两次所以要减掉），所以

$$f[i,\ j] = f[i,\ j-1] + f[i+1,\ j] - f[i+1,\ j-1] + [e[i,\ j] \ge m]$$

时间复杂度为 $\Theta(n^2 + q)$。

2.3 算法三

注意到直接处理询问不易，考虑将每次询问的答案拆开。

可以发现，如果 $[i,\ j]$ 不乏味，那么 $[i-1,\ j]$ 也不乏味，那么可以对于每个右端点 j，求出 e_j 表示最大的 i 使得 $[e_j,\ j]$ 是不乏味的。记 $S(n) = \sum_{i=1}^{n} i = n(n+1)/2$，那么询问的答案为

$$\begin{aligned}
&\sum_{j=k}^{l}[e_j \ge k]\sum_{i=k}^{e_j}(j-i) \\
=&\sum_{j=k}^{l}[e_j \ge k]\left(j(e_j-k+1) - \sum_{i=k}^{e_j} i\right) \\
=&\sum_{j=k}^{l}[e_j \ge k]\left(je_j - j(k-1) - S(e_j) + S(k-1)\right) \\
=&\sum_{j=k}^{l}[e_j \ge k]je_j - (k-1)\sum_{j=k}^{l}[e_j \ge k]j - \sum_{j=k}^{l}[e_j \ge k]S(e_j) + S(k-1)\sum_{j=k}^{l}[e_j \ge k]
\end{aligned}$$

因为 $e_j \geq k$ 的条件已经包含了 $j \geq k$，所以只需查询 $j \leq l$ 且 $e_j \geq k$ 的所有 je_j 的和、j 的和、$S(e_j)$ 的和、j 的个数。这里有两个限制，可以通过排序解决一个限制，另一个用树状数组维护。可以将询问离线后按右端点 l 排序，然后从小到大枚举右端点 j，先将当前 j 的四个值加入到树状数组的 e_j 位置处（此时树状数组中存的是所有 $j' \leq j$ 的值），再处理所有 $l = j$ 的询问，即查询此时树状数组中所有 $\geq k$ 处的和。

还有一个问题是如何求出 e_j，可以使用双指针的方法。先预处理出所有 j 的向前、向后最近的与 a_j 相等的位置 u_j, v_j，然后从小到大枚举右指针 j，同时维护左指针 i 表示 $[i, j]$ 的颜色数恰好为 m 的位置（即 e_j），右指针 j 右移时，颜色数可能超过了 m，于是尝试将左指针右移，直到颜色数等于 m。因为左右指针都是单调右移，所以预处理的时间复杂度为 $\Theta(n)$。如果您不会双指针，可以采用二分，每次在可持久化线段树上查询区间内 $u_i < l$ 的个数来找到 e_j，时间复杂度为 $\Theta(n \lg^2 n)$，或者直接在可持久化线段树上二分，时间复杂度为 $\Theta(n \lg n)$。双指针和在线段树上二分都是常用技巧，在不同的题目中各有用途。

总时间复杂度为 $\Theta(q \lg q + n \lg n + q \lg n)$。

赤(red)

本题来自 codeforces 739E,加大了数据范围.
首先对一只猫不会扔两个及以上数量的同种食物。最优方案一定把食物用完.
每只猫对期望的贡献可以根据期望的线性性分开算的，不投喂，则这只猫贡献为 0;
只喂干脆面，贡献为 pi;只喂豆干，贡献为 qi;两种都喂，贡献为 pi+qi-pi*qi
算法 1:
对于每只猫，只有 4 种情况，所以我们大力枚举一下，单组数据 O(4^n),可以通过第 1 个测试点，期望得分 10 分。
算法 2:
对于第 2 个测试点，有刚好等于猫数的豆干或干脆面，那么我们把等于猫数的食品一只猫喂一个，考虑每只猫多喂另一种食品后能新得到的贡献，例如，第 i 只猫已经喂了一个豆干，再喂一包干脆面对期望的贡献为(1-qi)*pi,可以认为不喜欢豆干时干脆面才会新产生贡献。我们排个序就好了，单组数据 O(n+nlogn),结合算法 1,期望得分 20 分.
算法 3:
考虑 DP，定义 f[i][j][k]为前 i 只猫投喂 j 包干脆面和 k 包豆干时得到的最大期望猫数,转移时考虑第 i 只猫如何投喂，复杂度 O(nab),可以通过 1,2,3,4 号测试点，期望得分 40 分。
算法 4:
注意到第 5 个测试点的 a 和 b 都相当大，那么没有被投喂的猫就会相对较少，我们考虑对没有被投喂的猫进行 DP。不妨先给所有的猫都同时投喂两种食品，然后再收回 n-a 包豆干和 n-b 包干脆面,最小化期望猫数的损失即可。DP 和算法 3 类似，时间复杂度 O(n(n-a)(n-b))结合算法 3,期望得分 50 分。

第 6 到 7 个测试点是为复杂度较优的算法准备的。
算法 5:
考虑网络流。给一只猫分配一包干脆面可以得到 pi 的收益，给一只猫分配一包豆干可以得

到 qi 的收益,但同时分配这两种食品将损失 pi*qi 的收益.如果不考虑 pi*qi 的损失，有一个很明显的费用流建图:建两个点表示豆干和干脆面，从原点向这两个点分别连费用为 0,流量等于对应食品数目的边，从干脆面向每只猫连流量为 1,费用为 pi 的边，从豆干向每只猫连流量为 1,费用为 qi 的边,从每只猫向汇点连流量为 2,费用为 0 的边。那么 pi*qi 的损失应当可以加到这个比较简单的模型的某一条边上。聪明的你一定想到了，我们把从猫连向汇点的边拆成两条，流量均为 1,但一条费用为 0,一条费用为-pi*qi,这样跑出来就是对的。
时间复杂度 O(费用流).
期望得分 70 分
算法 6:
cf 官方题解的方法，用数据结构维护贪心。~~可见这个题并没有考网络流~~

739E - Gosha is hunting

Let's divide Pokemons into 4 types: $0, A, B$ and AB depending on Balls that we throw to them. Let's sort them by u in descending order. Let's iterate over last Pokemon in which we throw Ultra Ball (his type is B or AB). Let i be the index of this Pokemon. It is not hard to prove that there are no Pokemons to the left of i that have type 0 and all Pokemons to the right of i have type 0 or A. Let's sort all Pokemons to the left of i by $(1 - p) * u$ in descending order. Let's iterate last Pokemon of type AB. Let j be his index. We can prove that to the left of j every Pokemon has type AB or B (Let's call this group of Pokemons X). Between j and i every Pokemon has type A or B (will call them Y). To the right of i only 0 and A (will call them Z). We know that we throw an Ultra Ball to every Pokemon in X. So let's add to our answer sum of u of all Pokemons in X. Number of Pokemons in Y of type B equals to the difference between b (number of Ultra Balls) and size of X. Therefore we know number of Pokemons in Y of type A and we know how many Poke Balls are left for X and Z summarily. Not hard to prove that in group Y we should throw Poke Balls to Pokemons with greatest $u - p$. Now we have to understand in which Pokemons in X and Z we should throw Poke Balls. If we throw Poke Ball to Pokemon from X, it adds to the answer $p * (1 - u)$, from Z - adds p. So we should throw Poke Balls to Pokemons with greatest values. When we iterate over j, each iteration one Pokemon moves from Y to X. We can keep structure that can add and delete one element, find minimum and keep sum of all elements in the structure. For example, treap or map (in c++). Let's keep 2 such structures: for calculating answer for Y and for calculating answer for throwing Poke Balls to Pokemons in X and Z. The complexity - O($n^2 \cdot logn$).

期望得分 70 分
算法 7
这是一种被称作 Wqs 二分(wqs=王钦石)的黑科技。熟悉 wqs 二分的选手可以直接去看算法 8 了.
假设看这里的选手没有听说过 wqs 二分.
考虑如何优化算法 3 的 O(n^3)DP.
我们首先不考虑 b 的限制,假设豆干可以任意使用，定义 f[i][j]表示前 i 只猫使用 j 包干脆面和若干包豆干得到的最大收益。直接这么 DP 会 n 只猫都用豆干，可能会超出 b 的限制。
为了减少豆干的使用,我们可以假定使用一个豆干需要额外付出 cost 的代价(也就是在转移的时候如果一只猫用了豆干,它对期望值的贡献要减去 cost).在 DP 的时候,求解出 f[i][j]的最大值,并记录最优方案中豆干使用的数目 x,那么此时真实的期望是 f[i][j]的最优值加上 x*cost,这个结果必然也是用了 x 个豆干时能够得到的最优结果.
但是,如果我们随便假定一个 cost 去跑 DP,得到的方案并不一定把所有豆干都用完,我们需要一个能把所有豆干都用完的 cost 的值.
显然豆干使用量随着 cost 的变化是单调变化的,我们可以二分 cost 的数值.
时间复杂度 O(n^2logn)
期望得分 70 分.
算法 8(满分算法)
我们还可以做到 O(n$\log^2$n)，具体做法在算法 7 的基础上,wqs 二分套 wqs 二分，把 DP 再减少一维。假设用豆干有 cost1 的额外花费,用干脆面有 cost2 的额外花费.f[i]表示前 i 只猫的最大收益.然后外层二分 cost1 内层二分 cost2.
看标程理解一下吧.

这个做法当然不是 lrd 想出来的啦.
出处:http://codeforces.com/blog/entry/49691
出题人的预测:NOIP 水平的选手拿到本题的前 50 分就不错了.这 50 分应当还是有区分度的.
水平稍高的选手可能打出一种复杂度较好的算法拿到 70 分.满分做法应当是相当难想到的.